

Session 5 : Intégration numérique 101 - Méthode d'Euler

Objectifs de la session

- Comprendre pourquoi l'ordinateur discrétise le temps (frames, pas Δt).
- Apprendre la méthode d'Euler pour intégrer position et vitesse.
- Introduire la série de Taylor comme fondement des méthodes d'intégration.
- Identifier les limites d'Euler et quand l'utiliser malgré tout.

💡 Comment l'ordinateur prédit le futur

Dans nos simulations, le temps n'est pas continu. L'ordinateur calcule le monde image par image (souvent 60 FPS, soit $\Delta t \approx 0.016$ s). Les équations physiques continues (dérivées) doivent devenir des instructions discrètes (additions).

1. La Méthode d'Euler

L'Algorithme

Pour chaque pas de temps Δt :

1. Calculer l'accélération : $\vec{a}_n = \frac{\vec{F}}{m}$
2. Mettre à jour la vitesse : $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$
3. Mettre à jour la position : $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \cdot \Delta t$

Intuition : On suppose que la vitesse est constante pendant tout le pas de temps.

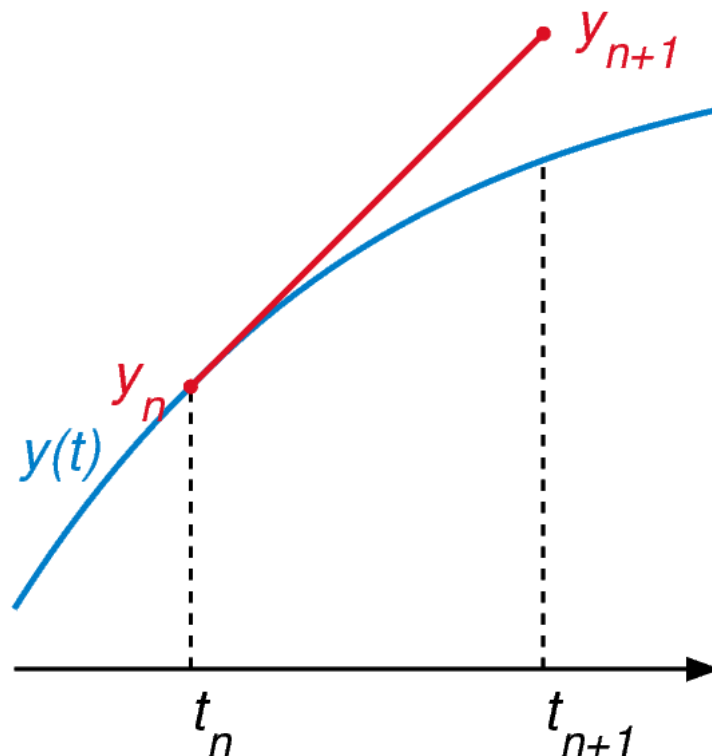


Figure 1: La méthode d'Euler : on avance pas à pas en supposant la vitesse constante pendant chaque Δt .

2. D'où ça vient ? La dérivée discrète

La vitesse est la dérivée de la position :

$$\vec{v}(t) = \frac{d\vec{r}}{dt} \approx \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}$$

En isolant le terme futur :

$$\underbrace{\vec{r}(t + \Delta t)}_{\text{Futur}} \approx \underbrace{\vec{r}(t)}_{\text{Présent}} + \underbrace{\vec{v}(t) \cdot \Delta t}_{\text{Pas}}$$

3. La Série de Taylor

Approximation d'une fonction

La série de Taylor dit qu'une trajectoire peut être reconstruite en additionnant ses dérivées successives :

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + \frac{1}{6}\vec{j}(t)\Delta t^3 + \dots$$

Euler ne garde que le premier terme. C'est une méthode du **premier ordre**.

Pourquoi Euler est une approximation ?

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n\Delta t + \underbrace{O(\Delta t^2)}_{\text{Erreur}}$$

4. Conséquences pratiques

- Euler ignore la courbure de la trajectoire (l'accélération) à l'intérieur du pas.
- Il tend à "dérapier" vers l'extérieur des virages ou à gagner de l'énergie (voir Figure 1).
- Malgré cela, il reste simple et rapide, et fonctionne quand l'accélération change (vent, ressorts, collisions).

Euler (Ordre 1)	Intégration exacte (Ordre 2)
$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n\Delta t$ <i>Prend la pente au début et trace une ligne droite.</i>	$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n\Delta t + \frac{1}{2}\vec{a}\Delta t^2$ <i>Prend en compte la courbure.</i>
Facile à coder. Rapide.	Exact pour la gravité constante.

Table 1: Comparaison pour un pas de temps Δt

5. Le code : euler_101.js

Nous allons maintenant décortiquer le fichier `examples/euler_101.js` : une simulation 3D d'une balle qui tombe et rebondit, propulsée par la méthode d'Euler. Le code est divisé en quatre couches : **l'état physique**, **l'initialisation Three.js**, **le cœur du moteur physique**, et **la boucle d'animation**.

5.1. L'état physique (variables globales)

Pourquoi des vecteurs mutables ?

Three.js fournit la classe `Vector3`. Au lieu d'en recréer un à chaque frame (`new THREE.Vector3(...)`), on en crée **un seul** qu'on modifie en place avec `.copy()`, `.add()`, `.set()`. C'est beaucoup plus doux pour le **garbage collector** du navigateur — un réflexe essentiel en boucle de jeu.

Paramètres et état initial.

```
// --- Paramètres Physique ---
const mass = 1.0;
const radius = 0.5;
const gravity = new THREE.Vector3(0, -9.81, 0);

// État initial de la balle (Position, Vitesse)
const startPosition = new THREE.Vector3(-5, 5, 0); // En haut à gauche
const startVelocity = new THREE.Vector3(3, 0, 0); // Lance vers la droite
const position = startPosition.clone();
const velocity = startVelocity.clone();
const force = new THREE.Vector3();
const acceleration = new THREE.Vector3();
```

- `mass`, `radius` : constantes de l'objet. La masse apparaît dans la 2^e loi de Newton $\vec{a} = \frac{\vec{F}}{m}$.
- `gravity` : le vecteur d'accélération gravitationnelle $\vec{g} = (0, -9.81, 0)$ "m/s"².
- `startPosition` / `startVelocity` : l'état à **t=0**, conservé pour pouvoir réinitialiser la simulation.
- `position`, `velocity`, `force`, `acceleration` : les **quatre vecteurs vivants** qui évoluent à chaque frame. On les clone pour ne pas écraser les valeurs de départ.

💡 Les quatre vecteurs fondamentaux

Toute simulation newtonienne repose sur ces quatre quantités, mises à jour dans cet ordre exact à chaque pas de temps :

$$\vec{F} \rightarrow \vec{a} \rightarrow \vec{v} \rightarrow \vec{r}$$

Force $\vec{F}$, accélération $\vec{a}$, vitesse $\vec{v}$, position $\vec{r}$. C'est la **chaîne d'Euler**.

5.2. Initialisation Three.js (init)

La fonction `init()` construit la scène une seule fois au démarrage. Elle ne contient **aucune** physique — uniquement de la plomberie graphique.

Scène, caméra, rendu, lumières et sol.

```
function init() {
  scene = new THREE.Scene();
  scene.background = new THREE.Color(0xbfd1e5);
  camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight,
0.1, 100);
  camera.position.set(0, 5, 15);

  renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.shadowMap.enabled = true;
  document.body.appendChild(renderer.domElement);
```

```

scene.add(new THREE.HemisphereLight(0xffeeee, 0xaaccaa));
const light = new THREE.DirectionalLight(0xFFFF22, 1);
light.position.set(5, 10, 7);
light.castShadow = true;
scene.add(light);

const floor = new THREE.Mesh(
  new THREE.PlaneGeometry(20, 20),
  new THREE.MeshStandardMaterial({ color: 0xaaccaa })
);
floor.rotation.x = -Math.PI / 2;
floor.receiveShadow = true;
scene.add(floor);
scene.add(new THREE.GridHelper(20, 20));

createBall();
setupGUI();
new OrbitControls(camera, renderer.domElement);
window.addEventListener('resize', onWindowResize, false);
renderer.setAnimationLoop(animate);
}

```

- **Scène & caméra** : la caméra PerspectiveCamera avec un champ de vision de 50°, placée en (0, 5, 15) pour voir la scène de biais.
- **Renderer** : WebGLRenderer dessine sur la page. antialias: true lisse les arêtes. shadowMap.enabled active les ombres portées.
- **Lumières** : une HemisphereLight (ciel + sol) pour l’ambiance, et une DirectionalLight (le “soleil”) qui projette des ombres.
- **Sol** : un PlaneGeometry couché par rotation.x = -Math.PI / 2 (rotation de -90° autour de l’axe X pour passer du plan XY au plan XZ). receiveShadow lui permet de recevoir l’ombre de la balle.
- **Lancement** : renderer.setAnimationLoop(animate) demande au navigateur d’appeler animate à chaque frame (≈ 60 FPS).

5.3. La balle et ses flèches (createBall)

Maillage + flèches de débogage.

```

function createBall() {
  ball = new THREE.Mesh(
    new THREE.SphereGeometry(radius, 32, 32),
    new THREE.MeshStandardMaterial({ color: 0x0077ff })
  );
  ball.castShadow = true;
  scene.add(ball);

  velArrow = new THREE.ArrowHelper(new THREE.Vector3(1, 0, 0), position, 1,
0x333333);
  scene.add(velArrow);

  accArrow = new THREE.ArrowHelper(new THREE.Vector3(0, 1, 0), position, 1,
0xAA8800);
  scene.add(accArrow);
}

```

- `ball` : un Mesh = une SphereGeometry (le rayon vient de notre variable physique) + un matériau bleu. `castShadow` projette son ombre sur le sol.
- `velArrow` / `accArrow` : deux ArrowHelper qui affichent en temps réel la **direction** et la **magnitude** de la vitesse (noir) et de l'accélération (jaune). Indispensables pour **voir** la physique — sans eux, Euler est une boîte noire.

5.4. Le cœur physique : `updatePhysics(dt)`

C'est ici que la méthode d'Euler prend vie. La fonction suit **exactement** les trois étapes de l'algorithme vu en section 1, plus une quatrième pour gérer le rebond.

Étape 1 — Calcul des forces.

```
function updatePhysics(dt) {
  // 1. Calcul des Forces
  force.set(0, 0, 0); // Reset

  // Ajout de la gravité (P = m * g)
  const gravityForce = gravity.clone().multiplyScalar(mass);
  force.add(gravityForce);
```

On **remet à zéro** la force à chaque frame (`force.set(0,0,0)`) — sinon les forces s'accumuleraient d'une frame à l'autre et la balle s'envolerait. Puis on ajoute le poids $\vec{P} = m \cdot \vec{g}$. Ici, seule la gravité agit ; dans un vrai jeu, on empilerait ici ressorts, vent, frottements, etc.

Étape 2 — Deuxième loi de Newton.

```
// 2. Deuxième Loi de Newton : a = F / m
acceleration.copy(force).divideScalar(mass);
```

On calcule $\vec{a} = \frac{\vec{F}}{m}$. `.copy(force)` recopie la force dans `acceleration`, puis `.divideScalar(mass)` divise par la masse. Avec $m = 1$, on a $\vec{a} = \vec{g}$, ce qui est cohérent : en chute libre, tout tombe à la même accélération.

Étape 3 — Intégration d'Euler.

```
// 3. Intégration d'Euler (Premier ordre)
// v_nouveau = v_actuel + a * dt
velocity.addScaledVector(acceleration, dt);

// p_nouveau = p_actuel + v * dt
position.addScaledVector(velocity, dt);
```

Ce sont **exactement** les deux formules de la définition de la section 1 :

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$$

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \cdot \Delta t$$

Subtilité : la vitesse utilisée pour la position

Remarquez qu'on met à jour la vitesse **avant** la position, et qu'on utilise la **nouvelle** vitesse $\vec{v}_{n+1}$ pour intégrer la position. C'est la variante **semi-implicite** (ou **symplectique**) d'Euler. Elle est légèrement plus stable que l'Euler classique (qui utiliserait $\vec{v}_n$) et conserve mieux l'énergie sur le long terme. C'est le choix par défaut dans la plupart des moteurs de jeu.

Étape 4 — Collisions avec le sol.

```
// 4. Gestion des Collisions (Sol)
if (position.y < radius) {
    position.y = radius;
    velocity.y = -velocity.y * params.restitution;
    velocity.x *= 0.95;
    velocity.z *= 0.95;
}
}
```

- **Test** : si le centre de la balle descend sous l'altitude `radius`, le bas de la sphère touche le sol.
- **Correction de position** : on remonte la balle à `position.y = radius` pour éviter qu'elle ne s'enfonce ou tremble sous le plan.
- **Rebond** : on inverse la composante verticale de la vitesse et on la multiplie par le coefficient de restitution `e` (`params.restitution`). Avec $e = 1$ la balle rebondit à la même hauteur ; avec $e = 0$ elle s'écrase ; à $e = 0.8$ elle perd 20 % d'énergie à chaque rebond.
- **Frottement tangentiel** : on amortit légèrement les composantes horizontales ($\times 0.95$) pour simuler le frottement au contact — sinon la balle glisserait indéfiniment.

💡 Pourquoi un test simple suffit ici

On teste uniquement `position.y < radius` parce que le sol est un plan infini à $y = 0$. Pour des collisions entre **deux sphères**, le test deviendrait $\|\vec{r}_A - \vec{r}_B\| < r_A + r_B$ — c'est-à-dire que la distance entre les centres est inférieure à la somme des rayons. On verra ça en Session 6.

5.5. Synchronisation visuelle (`updateVisuals`)

La physique calcule des nombres ; Three.js affiche des objets. `updateVisuals` fait le pont.

Copier la physique vers le visuel.

```
function updateVisuals() {
    ball.position.copy(position);

    velArrow.position.copy(position);
    velArrow.setDirection(velocity.clone().normalize());
    velArrow.setLength(velocity.length() * 0.3, 0.2, 0.1);

    accArrow.position.copy(position);
    if (acceleration.lengthSq() > 0.0001) {
        accArrow.setDirection(acceleration.clone().normalize());
        accArrow.setLength(acceleration.length() * 0.3, 0.2, 0.1);
    }
}
```

- `ball.position.copy(position)` : on **copie** le vecteur physique dans le mesh — jamais d'affectation directe (`ball.position = position` casserait la référence interne de Three.js).
- **Flèche vitesse** : `.normalize()` donne la direction (vecteur unitaire), `.length()` donne la magnitude. La longueur de la flèche est proportionnelle à $\|\vec{v}\|$, donc la flèche **grandit** quand la balle accélère.
- **Flèche accélération** : on vérifie `lengthSq() > 0.0001` avant de normaliser pour éviter une division par zéro (un vecteur nul n'a pas de direction). Ici l'accélération est constante (gravité), donc la flèche pointe toujours vers le bas avec la même longueur.

5.6. La boucle d'animation (`animate`)

Le pouls de la simulation.

```
function animate() {
  const dt = clock.getDelta();
  const safeDt = Math.min(dt, 0.1) * params.timeScale;
  updatePhysics(safeDt);
  updateVisuals();
  renderer.render(scene, camera);
}
```

! Le `dt` est le cœur d'Euler

`clock.getDelta()` renvoie le temps écoulé **en secondes** depuis la dernière frame — c'est notre Δt . Sans lui, la simulation tournerait à des vitesses différentes selon la machine (144 Hz vs 60 Hz vs 30 Hz). Avec `dt`, la physique est **indépendante du framerate** : la balle met le même temps à tomber sur un vieux portable que sur un écran de gamer.

- **Clamp** : `Math.min(dt, 0.1)` plafonne Δt à 0,1 s. Si l'utilisateur change d'onglet, le navigateur suspend la boucle ; au retour, `getDelta()` renverrait un Δt énorme (plusieurs secondes) qui ferait **exploser** la simulation (la balle traverserait le sol). Le clamp évite ce piège classique.
- **timeScale** : `params.timeScale` permet de ralentir (0,5×), accélérer (2×) ou mettre en pause (0×) la simulation sans toucher au code physique — utile pour observer un phénomène.
- **Ordre** : `updatePhysics` → `updateVisuals` → `render`. On calcule l'état, on le reflète sur les objets, puis on dessine. Cet ordre garantit que l'image affichée correspond **toujours** à l'état physique le plus récent.

6. Résumé : la chaîne complète

De la force à l'image, en une frame

À chaque image, `animate` déclenche cette cascade :

1. `clock.getDelta()` → mesure Δt réel.
2. `updatePhysics(dt)` :
 - Calcule $\vec{F}$ (gravité).
 - $\vec{a} = \frac{\vec{F}}{m}$.
 - $\vec{v}_{n+1} = \vec{v}_n + \vec{a} \cdot \Delta t$ (Euler).
 - $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \cdot \Delta t$.
 - Teste et résout la collision avec le sol.
3. `updateVisuals()` → copie $\vec{r}$ et $\vec{v}$ vers le mesh et les flèches.
4. `renderer.render()` → dessine la frame à l'écran.

C'est exactement le schéma de la Figure 1, traduit en JavaScript. Tout l'art d'un moteur de jeu tient dans cette boucle, exécutée 60 fois par seconde.